

Sviluppo di un sistema RAG aziendale e valutazione di LLM locali

Francesco Romeo

Anno Accademico 2025/2026

Contesto e obiettivo del lavoro

La tesi si basa su una realizzazione pratica che si è svolta all'interno dell'azienda Intré S.r.l., vale a dire fare rendere più accessibili le informazioni generate nel corso del tempo dai gruppi di apprendimento aziendali. La piattaforma aziendale utilizzata per l'implementazione di questi processi si chiama i3Guild. Tali dati, nel loro insieme, formano una memoria di lavoro utile, ma non sempre di facile consultazione. Le informazioni sono distribuite tra database relazionali, note testuali, metadati e contenuti multimediali, come video, blog e pdf; recuperarle richiede spesso familiarità con il dominio oppure una ricerca manuale poco efficiente.

In questo progetto viene ideato, implementato e testato un sistema *Retrieval-Augmented Generation* (RAG) integrato direttamente i3Guild: il contributo del lavoro consiste dunque nell'integrare componenti esistenti in un prototipo enterprise on-premise, adatto a rispondere a domande in linguaggio naturale sul corpus delle gilde. La soluzione proposta dovrà soddisfare dei requisiti specifici: la privacy dei dati dell'azienda deve essere garantita, il prototipo deve funzionare su hardware locale, non dedicato, e l'architettura deve inserirsi nell'applicazione esistente mantenendo separati database applicativo, indice vettoriale, retrieval e generazione.

La scelta del RAG dipende direttamente da un limite noto degli LLM. Un modello preaddestrato è in grado di produrre testo coerente ma non dispone dei dati privati e aggiornati di un'organizzazione. Se non riceve fonti esplicite, può formulare risposte plausibili ma difficili da verificare. Nel sistema realizzato, invece, la domanda dell'utente attiva una fase di recupero: i documenti pertinenti vengono cercati nella base di conoscenza e inseriti nel prompt del modello generativo. La risposta dipende quindi sia dal modello sia dal contesto recuperato. In questo modo la conoscenza può essere aggiornata modificando il corpus o l'indice, senza intervenire sui pesi del modello garantendo in oltre che le risposte fornite siano sempre verificabili.

La decisione di utilizzare il RAG deriva dalla natura stessa degli LLM. Un qualsiasi modello pretrained riesce ad elaborare dei testi coerenti, tuttavia, esso non conosce i dati interni e aggiornati dell'azienda: sarà quindi in grado di creare delle risposte ragionevoli, tuttavia difficile da verificare. Al contrario, nel caso del nostro sistema grazie al RAG la richiesta

utente avvia il processo di retrieval: i documenti necessari vengono individuati all'interno della base di conoscenze e inseriti nel prompt del modello generativo. Questo significa che la risposta sarà influenzata sia dal modello, sia dal contesto recuperato. In questo modo per ottenere risposte puntuali sarà sufficiente aggiornare la base di conoscenza mentre non viene toccato il modello.

Fondamenti tecnici e requisiti

La prima parte della tesi descrive i concetti tecnici necessari per comprendere il prototipo realizzato. I Transformer sono l'elemento fondamentale sia nei modelli linguistici generativi, sia nei modelli di embedding. Attraverso la self-attention, il modello mette in relazione i token di una sequenza e costruisce rappresentazioni contestuali del testo. Questo meccanismo permette di eliminare diversi limiti delle architetture ricorrenti; tuttavia non risolve da solo il problema della conoscenza aziendale. Un LLM non collegato a fonti esterne rimane un sistema a conoscenza limitata: non consulta i dati interni, non li aggiorna e non offre, da solo, un metodo affidabile per verificare le informazioni prodotte.

I sistemi RAG riescono invece ad offrire un metodo per verificare le informazioni prodotte grazie ad un'efficace separazione tra le operazioni di ingestione e inferenza. La fase di ingestione permette di creare delle rappresentazioni idonee per la creazione degli embedding sia densi che sparsi e, alla fine, salvare queste rappresentazioni su una base dati vettoriale. La fase di inferenza avviene invece a richiesta utente. La richiesta viene convertita allo stesso spazio usato nella fase di ingestione e con l'aiuto dell'indice di retrieval vengono ottenuti i documenti più pertinenti alla query dell'utente, questi ultimi vengono in fine aggiunti al prompt generativo.

Lo scenario d'uso studiato riguarda il sostenere una proposta per la formazione delle nuove gilde. La persona che propone un determinato tema potrebbe avere bisogno di controllare se temi simili siano stati discussi precedentemente, quali risultati sono stati ottenuti, chi ha preso parte e quando si sono verificate delle proposte simili. Durante l'analisi del corpus si è stabilito che la Gilda è l'entità più adatta all'indicizzazione: ogni Gilda include descrizioni, obiettivi, rischi, risultati attesi o realizzati e collegamenti con partecipanti ed epoche. I metadati vengono tenuti a parte dagli elementi indicizzati ed utilizzati come filtri.

I requisiti del sistema derivano direttamente dalle considerazioni espresse sopra. Il prototipo deve recuperare informazioni dal corpus di gilde, formulare risposte in linguaggio naturale, conservare traccia del contesto usato, supportare filtri strutturati e integrarsi con il frontend già presente. Deve anche oltre che ridurre la dipendenza da servizi esterni. La soluzione prevede di usare PostgreSQL come sorgente autoritativa, Qdrant come database vettoriale, Haystack come orchestratore della pipeline di retrieval, FastAPI per esporre il microservizio RAG, Ollama per l'inferenza locale e infine Next.js come livello applicativo.

Progettazione e implementazione del sistema

L'architettura che viene utilizzata rispecchia il flusso dei dati. PostgreSQL è responsabile del salvataggio delle informazioni legate a i3Guild. La pipeline di embedding legge le

gilde, normalizza i campi di testo e crea i documenti in Haystack. In Qdrant si salvano le rappresentazioni dense e sparse degli embedding oltre che il payload strutturato. FastAPI mette a disposizione gli endpoint per recuperare ed utilizzare la chat, sia in modifica sincrona che streaming e per il retrieval dei documenti. L'applicazione Next.js utilizza il servizio attraverso gli specifici client. Grazie a questa separazione è possibile operare su ogni componente in modo indipendente ad esempio aggiornando il corpus dei dati o cambiando il modello di Ollama senza preoccuparsi che qualcosa possa smettere di funzionare nel modo desiderato.

Allo stage iniziale della pipeline d'ingestione, vi è la verifica della configurazione, della connessione al database PostgreSQL, della raggiungibilità di Qdrant e della coerenza dei parametri di embedding. Si passa poi all'estrazione delle informazioni relative alle gilde, alla pulizia dei campi e alla formazione, per ogni gilda, di un documento strutturato in sezioni identificabili. Come anticipato, per ogni gilda si procede all'indicizzazione del documento singolo, in quanto la lunghezza dei campi testuale, per cui la totalità delle gilde, è in grado di rientrare nelle caratteristiche del modello d'embedding adottato. Viene però fissato un valore soglia: ogni volta che i campi testuali superano questo valore, questi verranno divisi in chunk sovrapposti. Gli ID dei documenti sono deterministici in questo modo una reindicizzazione sovrascrive i punti esistenti in Qdrant invece di duplicarli.

La ricerca non consiste solo in un confronto vettoriale: prima della ricerca, il sistema prova a isolare il tema della domanda. Nel dominio i3Guild parole come "gilda", "fondare" o "consigli" sono poco informative; in una frase come "vorrei fondare una gilda sulla musica", il termine utile per il retrieval è "musica" è stato quindi realizzato un sistema, deterministico, per tentare di estrapolare il reale significato dietro ad ogni query inviata al sistema. In parallelo, il servizio riconosce alcuni vincoli strutturati espressi in linguaggio naturale, per esempio una specifica epoca, una modalità di partecipazione o la distinzione tra gilde individuali e di gruppo. Dopo il recupero, i risultati vengono deduplicati e selezionati con soglie dinamiche, in questo modo il modello non riceve mai documenti ridondanti o marginali.

La generazione avviene tramite Ollama, utilizzato come server locale. Il microservizio riceve un prompt con: istruzioni, domanda utente e documenti recuperati; poi invoca il modello e restituisce la risposta. Lo streaming NDJSON separa eventi di retrieval, token generati, warning ed errori. Il frontend può quindi distinguere le fasi della risposta e mostrare l'avanzamento in modo più reattivo. Sono stati introdotti anche controlli operativi, tra cui stima della memoria disponibile, gestione dei timeout, rate limiting e distinzione tra errori applicativi e dipendenze non raggiungibili.

Durante lo sviluppo è stato sperimentato anche un workflow agent-based per il supporto alla proposta delle nuove gilde. In tesi viene presentato come un cammino perseguito, ma non come parte centrale della soluzione finale. La motivazione è pragmatica: sullo stesso hardware, una strategia agent-based comporta una maggiore quantità di chiamate al modello e di conseguenza maggiori tempi di latenza che si traducevano in un prototipo inutilizzabile. Nell'ambito del sistema sviluppato, una RAG più tradizionale ha dimostrato di essere più stabile e facile da gestire. La componente agentica rimane quindi una possibilità futura, da riprendere solo se porta un beneficio misurabile e avendo a disposizione hardware

dedicato.

Valutazione dell'inferenza locale

La seconda parte della tesi si è concentrata sull'inferenza locale. Un sistema RAG può essere ben progettato, ma la sua utilità dipende anche dal modello generativo che produce la risposta finale. Il modello deve rientrare nella memoria disponibile, caricarsi in tempi accettabili, iniziare a generare senza ritardi e mantenere una qualità compatibile con il caso d'uso.

Le misurazioni sono state effettuate su un MacBook Pro con chip M1 Pro e 16 GB di memoria unificata. Per sfruttare al meglio l'hardware disponibile sono stati utilizzati strumenti di inferenza progettati specificamente per Apple Silicon, così da beneficiare dell'accelerazione offerta dal backend Metal. I modelli analizzati coprono una gamma compresa tra 3 e 14 miliardi di parametri e sono stati suddivisi in tre categorie: small, medium e large. Per i modelli di dimensioni ridotte è stato confrontato il formato baseline FP16 con le versioni quantizzate Q8, Q6 e Q4; per i modelli medi e grandi, invece, l'analisi si concentra principalmente sulle varianti quantizzate, poiché la compressione della memoria diventa indispensabile per rispettare il vincolo dei 16 GB di memoria unificata.

Le metriche utilizzate includono: Dimensione sul disco, Memory peak, Loading time, Time to first token, Decode throughput e End-to-end throughput. A queste metriche si associano i test di qualità automatici su sottoinsiemi piccoli di MMLU e GSM8K. I subset non hanno lo scopo di produrre una classifica generale dei modelli; vengono usati per rilevare degradazioni evidenti, per privilegiare una completa automazione del processo tali test accettano il rischio di problemi del formato e incompatibilità con il parser.

Sui modelli di dimensioni ridotte, Q4 si rivela essere il compromesso ottimale nel setup misurato, esso riduce la dimensione degli artefatti di circa il 70%, abbassa il picco di memoria di circa 67–70% e aumenta il throughput di decodifica di circa 2.6–2.8 volte rispetto a FP16. Nei controlli automatici non si osserva un degrado significativo. Q8 e Q6 restano opzioni più conservative, ma dai dati raccolti non mostrano un vantaggio qualitativo sufficiente a compensare il maggiore uso di memoria e gli svantaggi ad esso collegati, come ad esempio una minor finestra di contesto utilizzabile.

Per i modelli medi e grandi, la quantizzazione ha soprattutto una funzione di fattibilità: rende eseguibili configurazioni poco pratiche sul target hardware utilizzato. Qwen3 8B e Mistral 7B diventano utilizzabili in Q4 con throughput intorno a 32–35 token al secondo e picchi di circa 5 GB. Gemma 3 12B e Qwen3 14B sono eseguibili in Q4, ma il throughput scende intorno a 18 token al secondo e il time-to-first-token supera il secondo. Queste configurazioni dimostrano che l'esecuzione locale è possibile; non indicano, però, che siano sempre la scelta migliore per un'applicazione interattiva. In un sistema completo, memoria e risorse devono essere condivise tra molti altri processi.

Risultati, limiti e sviluppi futuri

Il primo risultato raggiunto è la conversione dei dati dell'i3Guild in una knowledge base interrogabile tramite linguaggio naturale, senza necessità di spostare i dati al di fuori dal perimetro aziendale. La seconda conclusione ha invece un carattere più sperimentale: l'uso dei modelli locali su hardware consumer, così come prevedibile, è realizzabile, ma richiede delle scelte ponderate per quanto riguarda la dimensione del modello, quantizzazione, memoria e latenza. Il punto centrale non è quindi il singolo componente utilizzato, ma la catena completa degli stessi.

Tuttavia, ci sono alcune limitazioni importanti. Il prototipo RAG non dispone ancora di una ground truth costruita insieme agli stakeholder aziendali; per questo motivo non esistono informazioni sistematiche di groundedness, completezza, utilità e allucinazioni sul corpus reale delle gilde. È importante notare anche che la sperimentazione dell'inferenza locale è stata eseguita su un singolo dispositivo e con un insieme di test di qualità ridotti. I risultati sono utili per il caso d'uso specifico preso in considerazione, ma non bisogna estenderli automaticamente ad altri setup hardware/runtime.

Le prospettive per lo sviluppo futuro dovrebbero in un primo momento concentrarsi sulle valutazioni. Si dovrebbe pertanto costruire un set di domande applicative sul domain i3Guild, insieme alle risposte prevedibili e ai documenti di riferimento che siano stati validati da persone interne alla stessa azienda. In base a ciò, si potranno valutare le versioni diverse della pipeline e stabilire il ruolo specifico di ciascun elemento della stessa.

I test sui modelli locali aprono un ulteriore punto di miglioramento, le misure raccolte isolano il modello generativo e chiariscono il ruolo della quantizzazione, ma non descrivono, da sole, il comportamento del sistema RAG completo. Una verifica successiva dovrebbe eseguire i modelli candidati dentro il flusso reale all'interno del contesto reale. Solo una prova di questo tipo può confermare se i profili emersi nei benchmark di quantizzazione restano validi nel prototipo integrato nel sistema reale.

La componente agentica potrà essere ripresa dopo aver consolidato il RAG di base. Un agente potrebbe aiutare nella proposta di nuove gilde, nella raccolta di requisiti o nel collegamento con iniziative passate, la sua adozione dovrebbe dipendere però da criteri verificabili: minore carico per l'utente, bozze di qualità migliore e costi computazionali sostenibili. In assenza di questi dati, aumentare la complessità dell'orchestrazione rischia di peggiorare un sistema che, nella forma classica realizzata risulta essere utilizzabile.